codegraph_mcp_preflight.sh — guide

Revision: 1 **Last modified:** 2026-09-26T12:59:10Z

Source: `constitution/scripts/codegraph/codegraph_mcp_preflight.sh` (173 lines). Tests: `constitution/scripts/codegraph/tests/test_codegraph_mcp_preflight.sh` (cases F1-F14) and the mutation harness `test_codegraph_mcp_preflight_mutations.sh` (mutants Y1-Y6). Citations below are `codegraph_mcp_preflight.sh:<line>` unless another file is named.

Overview

A pre-flight check that answers, with evidence and without touching the project's index, "can an agent safely use the CodeGraph MCP server here?" (header, lines 3-15). It runs six numbered checks and prints one line per check:

Id	What it checks	Source
P1	`.mcp.json` declares a `codegraph` server whose command is the sanctioned `codegraph_mcp.sh` (executable, with `fk_index_patch.py` beside it) and whose arguments are only `serve --mcp [--project DIR]`	lines 43-89
P2	No OTHER `.mcp.json` server entry launches the stock `codegraph`	lines 90-98
P3	Optional settings files carry no stock-codegraph hook or server. Runs only when `--settings` is given	lines 101-128
P4	The wrapper resolves a receipt-valid `mcpro1` runner for a throwaway fixture (may build it into the runner cache)	lines 142-148
P5	A fixture query is answered end to end through the wrapper AND the server holds the database read-only (descriptor modes read from `/proc` by the probe)	lines 149-158
P6	The real project has an index; a live writer on it is a WARN, any other refusal is a FAIL	lines 162-171

The real project is only ever `stat`-ed and passed to `codegraph_mcp.sh --dry-run`, which reads `/proc` and the lock file but never opens the database and never runs `codegraph` (header lines 14-15). The fixture for P4/P5 is created in a temporary directory with the pristine package (lines 131-140).

Prerequisites

- `bash`, `python3`, `git`, `sed`, `mktemp` (lines 40, 43, 135). The header also lists "a pristine codegraph 1.6.x package" (line 20): the fixture is indexed with `$HELIX_CODEGRAPH_SRC/bin/codegraph init` (lines 131-136).
- Files beside the script: `codegraph_mcp.sh`, `codegraph_mcp_probe.py`, `fk_index_patch.py` (lines 37, 80, 131).
- Linux `/proc` (through the wrapper and the probe).
- `--project` must be an existing directory (line 35).

Usage

```
codegraph_mcp_preflight.sh [--project DIR] [--mcp-json FILE] [--settings FILE]...
codegraph_mcp_preflight.sh -h | --help
```

Argument	Meaning	Source
`--project DIR`	Project to check. Default `$PWD`. Must exist, otherwise exit 2 (`project directory not found`)	lines 25, 28, 35
`--mcp-json FILE`	`.mcp.json` to inspect. Default `<project>/.mcp.json`. The path is opened as given (relative to the current directory, not to the project)	lines 29, 36, 48
`--settings FILE`	A settings file for P3. Repeatable. Without it P3 does not run and no `P3` line is printed	lines 30, 101
`-h`, `--help`	Print lines 2-26 of the script and exit 0	line 31
anything else	`unknown argument: <arg>` on stderr, exit 2. A flag without its value prints `usage: --<flag> ...` and exits 2	lines 28-30, 32

Examples

#	Command	Status
1	`codegraph_mcp_preflight.sh --help`	VERIFIED (ran; saw 25 lines, exit 0; the last four are shell code, see Findings)
2	`--bogus`, and `--settings` with no value, and `--project <missing dir>`	VERIFIED (ran; saw `unknown argument: --bogus`, `usage: --settings FILE`, `project directory not found`; exit 2 each)
3	`codegraph_mcp_preflight.sh --project <fixture project with a correct .mcp.json and an index>`	VERIFIED (ran on a throwaway fixture project inside the scratch area, with a scratch runner cache and fixture `HOME` / `CLAUDE_CONFIG_DIR`; saw `P1 PASS`, `P2 PASS`, `P4 PASS ... (1.6.0-mc-pro1)`, `P5 PASS`, `P6 PASS index present, no live writer, runner valid — the server would start`, `SUMMARY fail=0 warn=0`, exit 0)
4	Same, but `--project` is an empty directory	VERIFIED (ran; saw `P1 FAIL no .mcp.json at <dir>/.mcp.json`, `P2 FAIL no .mcp.json to inspect`, `P4 PASS`, `P5 PASS`, `P6 FAIL no CodeGraph index at <dir>/.codegraph/codegraph.db`, `SUMMARY fail=3 warn=0`, exit 1; the empty directory stayed empty)
5	A scratch directory holding a copy of this repository's `.mcp.json` and a `constitution` symlink, used as `--project`	VERIFIED (ran; saw `P1 PASS 'code-graph' server -> .../constitution/scripts/codegraph/code-graph_mcp.sh`, `P2 PASS`, `P4 PASS`, `P5 PASS`, and `P6 FAIL` only because the scratch directory has no index; exit 1)
6	`--settings <file whose hook matcher is mcp__code-graph__codegraph_explore>`	VERIFIED (ran; saw `P3 FAIL <file>: hook runs the stock codegraph: mcp__codegraph__codegraph_explore`, exit 1; see Edge cases)

#	Command	Status
7	`--settings <nonexistent file>`	VERIFIED (ran; saw `P3 PASS settings carry no stock-codegraph hook/server`, exit 0)
8	`--settings <file with invalid JSON>`	VERIFIED (ran; saw `P3 FAIL <file> unreadable (Expecting ',' delimiter: line 1 column 12 (char 11))`, exit 1)
9	Live writer on the fixture index (a process holding the database in `BEGIN IMMEDIATE`)	VERIFIED (ran; saw `P6 WARN live writer on the pr...`, exit 0 — a WARN, never a FAIL; also test case F7 in a scratch directory with a longer path)
10	`codegraph_mcp_preflight.sh` with no arguments from the repository root (checks the real project)	NOT RUN (P6 would pass the real `.codegraph` to the wrapper's `--dry-run`; deliberately not done against the live index)

Inputs

Input	Effect	Source
`HELIX_CODEGRAPH_SRC`	Pristine package used for the fixture. Default: `python3 -c 'import fk_index_patch as m; print(m.default_src())'` run from the script directory	line 131
`HELIX_CODEGRAPH_FIXTURE_BIN`	Binary used to `init` the fixture. Default `$SRC/bin/codegraph`. Header: "tests: the fixture writer"	lines 17, 132, 136
`HELIX_CODEGRAPH_RUNNER_DIR`	Runner cache used by the wrapper (see the wrapper's guide)	header line 17
`HELIX_CODEGRAPH_MCP_RUNNER`	Read by the wrapper the script calls; pins a runner (test F6/F10/F11 use it)	via `code-graph_mcp.sh:55`
`TMPDIR`	Where the temp directory `cg_mcp_pf.XXXXXX` is created	line 40

Outputs

- stdout: one line per check, `PREFLIGHT <id> <PASS|FAIL|WARN> <text>` (line 39), in the order P1, P2, [P3], P4, P5, P6, then `SUMMARY fail=<N> warn=<M>` (line 172).
- stderr: usage and argument errors only.

Exit codes

Code	Meaning	Source
0	No check failed (`FAILS == 0`); WARN lines do not affect the code	lines 39, 173
1	At least one `FAIL` line	line 173
2	Usage: unknown argument, missing flag value, or project directory not found	lines 28-32, 35

Edge cases

- `.mcp.json` **problems fail P1 and P2 together**: missing file, invalid JSON, or no `mcpServers` object print a `P1 FAIL` and a `P2 FAIL` and skip the rest of the file check (lines 49-55).
- **How P1 finds the wrapper:** if the command's basename is `bash` , `sh` or `env` and there are arguments, the first argument is treated as the script and the rest as its arguments; a relative script path is joined to the project directory; the path is resolved with `realpath` (lines 56-65, 75). This is why this repository's own entry (`command: bash` , `args: [constitution/scripts/codegraph/codegraph_mcp.sh]`) passes (example 5).
- **P1 allows only** `serve` , `--mcp` , `--project DIR` , `--project=DIR` as arguments; anything else is reported as `argument 'x' is not allowed` (lines 83-88; test F4 with `index`). The `--project` value is not checked.
- **P1 reports at most one of three wrapper problems** (wrong basename, not executable, no `fk_index_patch.py` beside it) because they are an `if/elif` chain (lines 76-81); argument problems are added to that reason.
- **P2 compares basenames only.** For every other server it takes the basename of `command` and, when `command` is `bash` / `sh` / `env` , the basename of the resolved first argument, and flags the entry only if one equals exactly `codegraph` (lines 56-65, 90-98). Entries such as `node .../codegraph.js` , `bash -c "codegraph ..."` , `npx @colbymchenry/codegraph` or a shim script named otherwise are NOT flagged (test F14 records `res=ppppp` , five misses; see Findings).

- **P3 only runs with `--settings`,** and only looks at (a) every string anywhere under the file's `hooks` key that contains `codegraph` (case-insensitive) and does not contain `codegraph_mcp`, and (b) `mcpServers` entries whose command basename is `codegraph` (lines 111-125). Consequences, all observed in examples 6-8: a hook MATCHER string like `mcp__codegraph__codegraph_explore` is reported as a stock-codegraph hook; a nonexistent settings file is skipped silently and P3 PASSes (lines 114-115); an unparsable file is a P3 FAIL (lines 116-117).
- **P4/P5 need a fixture.** If `$FIXBIN` is not executable or `init` fails, only `P5 FAIL fixture could not be created with <bin> (init failed: ...)` is printed and P4 is not printed (lines 136-140). If the package cannot be found, `SRC` is empty and `FIXBIN` becomes `/bin/codegraph` (lines 131-132); NOT RUN with an empty source.
- **P4 passes only if the wrapper's dry-run exits 0 and `runner_key` ends in `-mcpro1`** (lines 142-147). A receipt-valid runner is not proof of working: P5 is the check that catches a runner that passes P4 but launches the stock writer or fails its session (tests F10, F11).
- **P5 requires every observed database descriptor to be read-only and at least one to exist** (`modes and all(m == "r")`, lines 154-155).
- **P6 with a live writer is a WARN (exit stays 0) by design** (line 166); the wrapper's stderr is cut to 220 bytes (line 166), and any other wrapper exit is a FAIL (line 167).
- **Temp files.** The script removes its own directory on exit (line 40, checked: no `cg_mcp_pf.*` left over). It does NOT remove the empty `cg_mcp_err.XXXXXX` file that the wrapper leaves behind when it starts the server: one such file remained in `TMPDIR` after each run that reached P5 (observed: 0 files before, 1 after a single run).

Findings (things in the tool or its tests that do not match)

1. **The test file expects features the source does not have.** `tests/test_codegraph_mcp_preflight.sh` asserts a `P7` check (user-scope `~/.claude.json` / `$CLAUDE_CONFIG_DIR/.claude.json`, case F13, header line 13) and P2 classification "by the RESOLVED launch target" (case F14,

comment at test line 144). Neither exists in `codegraph_mcp_preflight.sh`: the string `P7` occurs only in the test file, and P2 is basename-only (lines 90-98). The header of the script itself lists only P1-P6. Ran the test suite in a scratch `TMPDIR`: F1-F6, F8-F12 PASS; F13 and F14 FAIL (`p7a=` empty; `res=ppppp`), so the suite exits 1 (`SUMMARY cases=14 failed=3`). Source and tests were added by the same commit (`b3a73cf`).

2. **The mutation harness cannot run.** It first requires the baseline test suite to pass and otherwise stops (`test_codegraph_mcp_preflight_mutations.sh:17-18`); ran it: `BASELINE FAILED (rc=1)`, no mutant was applied.
3. **F7 depends on path length.** P6 cuts the wrapper's stderr with `head -c 220` (bytes, line 166); the wrapper's message contains an em dash (`—`, three bytes) right after the database path. The message prefix is 49 bytes, so the em dash starts at byte `51 + <path length>`; with a 168- or 169-byte path the 220-byte cut lands inside that character. Observed with a 168-byte path (the same fixture, a fresh live writer: rc 0; the `P6 WARN` line ends with the two bytes `E2 80`, `iconv -f UTF-8` rejects the output, and the test's `grep` prints `binary file matches` so F7 fails); the 169-byte case is derived from the arithmetic, not run. The same case PASSes when `TMPDIR` is 7 characters longer.
4. **`--help` prints code.** The header block ends at line 22, but `sed -n '2,26p'` (line 31) prints lines 2-26, so the last four printed lines are `set -u`, `HERE=...`, `PROJECT=...`, `while ...`. (The sibling wrapper's help range, `2,32`, matches its header exactly.)
5. **Undocumented side effect.** The header's "Side effects" (line 19) omits the leftover `cg_mcp_err.*` file (see Edge cases; the cause is in `codegraph_mcp.sh:54,140`).

Internal behaviour

1. Parse arguments; resolve the project; default `.mcp.json` (lines 23-37).
2. Create the temp directory and its EXIT trap (line 40).
3. P1/P2: an embedded Python program parses the JSON (never greps it) and prints one line per check; the shell reads them through a process substitution so the FAIL/WARN counters survive (a piped `while` would lose them, comment on line 100).

4. P3 (only with `--settings`): a second embedded Python program (lines 101-128).
5. Build the fixture project and index it with the pristine package's `init -y` (lines 131-140).
6. P4: `codegraph_mcp.sh --project <fixture> --dry-run`; P5: `codegraph_mcp_probe.py --bin <wrapper> --project <fixture> --query compute_total` and a Python assertion over its JSON (lines 141-159).
7. P6: `stat` the real database; if present, `codegraph_mcp.sh --project <project> --dry-run` (lines 162-171).
8. Print `SUMMARY` and exit `0` iff no FAIL (lines 172-173).

Related

File	Relation
codegraph_mcp	The wrapper this script exercises (P4, P5, P6)
codegraph_mcp_probe	MCP client that produces the JSON P5 asserts on
`fk_index_patch.py`	Builds/finds the `mcpro1` runner; also gives the default package source (line 131)
`tests/test_codegraph_mcp_preflight.sh`	Cases F1-F14 (F7, F13, F14 currently fail, see Findings)
`tests/test_codegraph_mcp_preflight_mutations.sh`, `..._mutate.py`	Mutants Y1-Y6 against a copy of this script (blocked by the baseline failure)
owed_report	Reports whether this tool has tests and a guide

Last verified

2026-09-26T12:59:10Z — everything ran in a scratch area (fixtures, runner cache, fixture `HOME` and `CLAUDE_CONFIG_DIR`, scratch `TMPDIR`); the repository's real `.codegraph` was never passed to any command.

- `bash tests/test_codegraph_mcp_preflight.sh`: 11 PASS, 3 FAIL (F7, F13, F14), `SUMMARY cases=14 failed=3`.
- `ONLY=F7 bash tests/test_codegraph_mcp_preflight.sh` with a longer `TMPDIR`: PASS.
- Re-run 2026-09-26 (second verification pass, scratch `TMPDIR` of a different length): F7 PASSED, F13 and F14 FAILED, `SUMMARY cases=14 failed=2` (confirms F7 is path-length dependent, Finding 3; F13/F14 fail deterministically). `--help` printed 25 lines, last four shell code (Finding 4). 21 `cg_mcp_err.*` files remained in `TMPDIR` after the suite (Finding 5).
- `bash tests/test_codegraph_mcp_preflight_mutations.sh`: `BASELINE FAILED (rc=1)`.
- The direct invocations in the examples table (help, usage errors, correct project, empty project, this repository's `.mcp.json` shape, three `--settings` cases, live writer).